

SENAI FATESG

Análise e Desenvolvimento de Sistemas — 3º Semestre
Arquitetura e Projeto de Software

ADR-004

Implementação dos Domínios Mecânico e Usuário Backend (Spring Boot) + Frontend (Angular)

Status: Em vigor **Data:** 01/04/2025 **Autor:** Cassiano Abreu

Caio Abreu | Cassiano Abreu | Gabriel Naoki | Wyllian Mariano
SENAI FATESG — Goiânia, GO — 01 de abril de 2025

Contexto

O frontend Angular já possuía páginas standalone e services configurados para os domínios Mecânico (/mecanicos) e Usuário (/usuarios), porém esses endpoints não existiam no backend Spring Boot. Isso causava erros 404 capturados pelo `HttpErrorInterceptor` em tempo de execução, impedindo o funcionamento das telas de mecânicos e bloqueando a criação da tela de usuários.

Adicionalmente, a ausência desses domínios bloqueava a atividade 4, que depende de `mecanicoResponsavelId` e `usuarioResponsavelId` para alinhar os campos da `OrdemServico` entre frontend e backend.

Decisão

Implementar os domínios Mecânico e Usuário de forma completa tanto no backend quanto no frontend, seguindo rigorosamente os padrões já estabelecidos nas versões anteriores.

No backend: seguir o padrão genérico do ADR-003 (`Model`, `DTO`, `IRepository`, `Repository`, `IService`, `Service`, `IValidation`, `Validation`, `IMapper`, `Mapper`, `Controller`) em pacotes isolados dentro de `business.*`.

No frontend: seguir o padrão dos domínios existentes (interface `TypeScript`, `service` herdando `ApiBaseService`, componente standalone com formulário e listagem, rota registrada no `app.routes.ts`).

Todas as alterações organizadas em branch separado (`feat/dominio-mecanico`) com commits atômicos por responsabilidade, submetidos via Pull Request.

Implementação — Backend

Domínio Mecânico — br.edu.senai.fatesg.ads3.car_repair.business.mecanicos

Arquivo	Camada	Responsabilidade
MecanicoModel.java	Model / Entidade JPA	Tabela Mecanico — campos: nome, cpf, email, telefone, especialidade, crea
MecanicoDTO.java	DTO	Contrato da API — mesmos campos do Model
IMecanicoRepository.java	Interface Repository	Estende IGenericRepository<MecanicoModel>
MecanicoRepository.java	Repository	Classe vazia — implementação pelo Spring Data
IMecanicoService.java	Interface Service	Estende IGenericService<MecanicoModel>
MecanicoService.java	Service	beforeInsert: define ativo=true e dataHoraCriacao
IMecanicoValidation.java	Interface Validation	Estende IGenericValidation<MecanicoModel>
MecanicoValidation.java	Validation	Implementação das validações do domínio
IMecanicoMapper.java	Interface Mapper	Estende IGenericMapper<MecanicoModel, MecanicoDTO>
MecanicoMapper.java	Mapper	Conversão bidirecional Model <-> DTO
MecanicoController.java	Controller	Endpoint REST /mecanicos — CRUD completo por herança

Domínio Usuário — br.edu.senai.fatesg.ads3.car_repair.business.usuarios

Arquivo	Camada	Responsabilidade
UsuarioModel.java	Model / Entidade JPA	Tabela Usuario — campos: nome, login, email, perfil, senha_hash
UsuarioDTO.java	DTO	Contrato da API — expõe nome, login, email, perfil (sem senha_hash)
IUsuarioRepository.java	Interface Repository	Estende IGenericRepository<UsuarioModel>
UsuarioRepository.java	Repository	Classe vazia — implementação pelo Spring Data
IUsuarioService.java	Interface Service	Estende IGenericService<UsuarioModel>

UsuarioService.java	Service	beforeInsert: define ativo=true e dataHoraCriacao
IUsuarioValidation.java	Interface Validation	Estende IGenericValidation<UsuarioModel>
UsuarioValidation.java	Validation	Implementação das validações do domínio
IUsuarioMapper.java	Interface Mapper	Estende IGenericMapper<UsuarioModel, UsuarioDTO>
UsuarioMapper.java	Mapper	Conversão bidirecional Model <-> DTO — senha_hash não exposta no DTO
UsuarioController.java	Controller	Endpoint REST /usuarios — CRUD completo por herança

Implementação — Frontend Angular

Domínio Mecânico — já existia, integração ativada

O componente de mecânicos já estava implementado no projeto. Com a criação do endpoint `/mecanicos` no backend, a integração passou a funcionar sem nenhuma alteração no frontend.

- `MecanicosComponent` — formulário com nome, especialidade e telefone; listagem em tabela
- `MecanicosService` — herda `ApiBaseService`; métodos `listar()` e `adicionar()`
- `Mecanico` (interface) — campos: `id`, `nome`, `especialidade`, `telefone`
- Rota `/mecanicos` — já registrada no `app.routes.ts`

Domínio Usuário — criado do zero

O domínio Usuário foi integralmente implementado no frontend seguindo o mesmo padrão dos demais domínios:

Arquivo	Tipo	Descrição
<code>usuario.ts</code>	Interface TypeScript	Campos: <code>id</code> , <code>nome</code> , <code>login</code> , <code>email</code> , <code>perfil</code> (union type)
<code>usuarios.service.ts</code>	Service Angular	Herda <code>ApiBaseService</code> — métodos <code>listar()</code> e <code>adicionar()</code>
<code>usuarios.component.ts</code>	Componente	Formulário com validação, listagem e integração com service
<code>usuarios.component.html</code>	Template	Formulário com select de perfil (administrativo/atendente/gestor)
<code>usuarios.component.css</code>	Estilo	Estilização dos erros de validação

Alterações nos arquivos globais do Angular:

- `app.routes.ts` — adicionada rota `/usuarios` apontando para `UsuariosComponent`
- `app.html` — adicionado link de navegação para `/usuarios` no menu principal

Justificativa

- Seguir o padrão genérico do ADR-003 garante consistência com os demais domínios e minimiza risco de erros
- Branches separados e commits atômicos por responsabilidade garantem rastreabilidade das mudanças
- A senha_hash não é exposta no `UsuarioDTO` por segurança — separação entre Model (banco) e DTO (API) aplicada
- O campo `perfil` usa union type no TypeScript alinhado com o ENUM do PostgreSQL

- A implementação frontend completa garante que o domínio Usuário funcione de ponta a ponta
- Nenhum dado simulado (mock) foi utilizado — a diretriz do projeto foi seguida rigorosamente

Alternativas Consideradas

- Criar endpoints sem as camadas completas (apenas Controller) — descartado por quebrar o padrão arquitetural do ADR-003
- Usar mock/stub no frontend enquanto o backend não estava pronto — descartado pela diretriz oficial do projeto
- Implementar autenticação JWT completa nesta versão — adiado para manter o escopo da atividade controlado
- ENUM Java para o campo perfil no Model — adiado para uma versão futura; String foi adotado por simplicidade

Consequências

- Os endpoints /mecanicos e /usuarios estão disponíveis no backend, resolvendo os erros 404 do frontend
- A tela de Usuários está funcional de ponta a ponta — formulário, validação, listagem e integração com a API
- A atividade 4 (alinhamento da OrdemServico com mecanicoResponsavelId e usuarioResponsavelId) fica desbloqueada
- 22 arquivos adicionados ao backend + 7 ao frontend — todos seguindo os padrões estabelecidos nos ADRs anteriores
- O domínio Usuário ainda não possui autenticação real — a senha_hash é persistida mas o mecanismo de login será implementado em versão futura
- O campo perfil do Usuário é String no backend — uma evolução futura pode convertê-lo para ENUM Java alinhado com o tipo PostgreSQL

Atualização v5.0 — Integração Angular Completa

Contexto

Com os domínios Mecânico e Usuário funcionando no backend (v4.0), o frontend Angular precisava ser elevado ao mesmo nível de qualidade: o design estava genérico, sem aderência ao HTML estático aprovado pela equipe, e as operações de edição e exclusão não estavam implementadas em nenhum domínio. Além disso, o feedback visual ao usuário (mensagens de sucesso/erro) era esteticamente inadequado.

Decisão

Integrar o design do HTML estático aprovado diretamente nos componentes Angular existentes, implementar operações de edição (PUT) e exclusão (DELETE) em todos os domínios, e substituir o sistema de mensagens por um toast flutuante com estética consistente. Todas as alterações organizadas em duas branches separadas com Pull Requests.

Implementação — Frontend Angular v5.0

Design Integrado (branch: feat/frontend-carrepair-ui-v2)

O design do HTML estático aprovado foi transposto integralmente para os componentes Angular. CSS variables globais (--p1, --accent, --bg, --surface) definidas em styles.css. Navbar com fundo azul escuro (#1a2744) e logo com destaque dourado. Hero com mock do sistema na página inicial. Cada página interna possui header escuro, botão dourado para novo registro, tabela estilizada com badges de status e formulário colapsável com animação fadeDown. Todos os 6 domínios (clientes, veículos, mecânicos, serviços, ordens de serviço, usuários) consomem a API real na porta 9081 — nenhum dado simulado. Validação de CPF com algoritmo Luhn idêntico ao backend, e mensagens de erro do GlobalExceptionHandler exibidas ao usuário.

Editar e Excluir + Toast (branch: feat/frontend-edit-delete-toast)

Botões Editar e Excluir adicionados na coluna Ações de todas as tabelas. Editar preenche o formulário com os dados do registro e executa PUT `{endpoint}/{id}` ao salvar. Excluir solicita confirmação via dialog nativo e executa DELETE `{endpoint}/{id}`. O ApiService foi corrigido para aceitar ResponseEntity<String> do backend (responseType: 'text'). O MensagensComponent foi reescrito como toast flutuante: position fixed no canto superior direito, borda colorida por tipo (verde/vermelho), animação toastIn, ícone ✓/X em círculo. Correção da tela de Ordens de Serviço: nomes de cliente, veículo e mecânico resolvidos via forkJoin, tabela com overflow-x para caber na tela.

Justificativa

O CRUD completo (criar, listar, editar, excluir) é requisito fundamental de um sistema de gestão — a ausência de editar/excluir deixava o sistema incompleto para uso real. A integração do design aprovado garante consistência visual entre a documentação e o produto entregue. O toast substitui a barra de notificação invasiva por um componente não-bloqueante, seguindo padrões modernos de UX. Branches separadas por responsabilidade mantiveram a rastreabilidade das mudanças.

Alternativas Consideradas

Reescrever o frontend do zero — descartado por desperdiçar o trabalho já existente nos services e rotas. Usar biblioteca de UI externa (Angular Material, PrimeNG) — descartado para manter controle total sobre o visual e evitar dependências desnecessárias. Modal customizado de confirmação — adiado em favor do dialog nativo do browser para simplicidade. Soft delete via campo ativo=false no PUT — descartado; o DELETE do GenericController já implementa exclusão lógica no backend.

Consequências

O frontend está completo e funcional de ponta a ponta — todos os 6 domínios com CRUD completo integrado à API real. O design é fiel ao HTML estático aprovado. Toast notifications melhoram a experiência do usuário sem bloquear a interface. Dois PRs abertos (feat/frontend-carrepar-ui-v2 e feat/frontend-edit-delete-toast) aguardam merge pelo Caio. A próxima evolução natural é a implementação de autenticação JWT e paginação com filtros nas listagens.